

16. Madni, A.M. and M. Sievers, System of Systems Integration: Key Considerations and Challenges. *Systems Engineering*, 2014. 17(3): p. 330-347.17.
17. Shadab, N., A.U. Kulkarni, and A. Salado, Shifting Paradigms in Verification and Validation of AI-Enabled Systems: A Systems-Theoretic Perspective, in *Systems Engineering and Artificial Intelligence*, W.F. Lawless, et al., Editors. 2021, Springer International Publishing: Cham. p. 363-378.
18. Shadab, N., et al. Product Herding for Intelligent Systems. in *Conference on Systems Engineering (CSER)*. 2023. Hoboken, NJ, USA.
19. INCOSE, Systems Engineering Vision 2035. 2023.
20. Wymore, A.W., A mathematical theory of systems engineering: The elements. 1967, New York: Wiley.
21. Warfield, J.N. and J.D. Hill, A unified systems engineering concept. Vol. Monograph 1. 1972, Columbus: Batelle Memorial Institute.
22. Mesarovic, M.D. General systems theory and its mathematical foundation. in *IEEE Systems Science and Cybernetics Conference*. 1967. Boston, MA.
23. Salado, A. and A.U. Kulkarni, An Assessment of the Adequacy of Common Definitions of the Concept of System. *INCOSE International Symposium*, 2021. 31(1): p. 510-521.
24. Sheard, S.A., TWELVE SYSTEMS ENGINEERING ROLES. *INCOSE International Symposium*, 1996. 6(1): p. 478-485.
25. INCOSE, Systems Engineering Competency Framework. 2018.
26. GAO, DHS Annual Assessment: Major Acquisition Programs Are Generally Meeting Goals, but Cybersecurity Policy Needs Clarification. 2023.
27. GAO, Space Acquisitions: DOD Faces Significant Challenges as it Seeks to Accelerate Space Programs and Address Threats. 2019.
28. Henderson, K. and A. Salado, Value and benefits of model-based systems engineering (MBSE): Evidence from the literature. *Systems Engineering*, 2021. 24(1): p. 51-66.
29. Henderson, K. and A. Salado, Is CAD A Good Paradigm for MBSE? *INCOSE International Symposium*, 2021. 31(1): p. 144-157.
30. Henderson, K., T. McDermott, and A. Salado, MBSE adoption experiences in organizations: Lessons learned. *Systems Engineering*. n/a(n/a).
31. Henderson, K. and A. Salado, The Effects of Organizational Structure on MBSE Adoption in Industry: Insights from Practitioners. *Engineering Management Journal*, 2023. In press.
32. Salado, A., 5.5.2 Efficient and Effective Systems Integration and Verification Planning Using a Model-Centric Environment. *INCOSE International Symposium*, 2013. 23(1): p. 1159-1173.
33. Gregory, J. and A. Salado, Model-Based Verification Strategies Using SysML and Bayesian Networks, in *Conference on Systems Engineering Research (CSER)*. 2023: Hoboken, NJ, USA.
34. Hecht, M. and J. Chen. Use of SysML for Quantitative System Reliability and Availability Analysis. in *2022 Annual Reliability and Maintainability Symposium (RAMS)*. 2022.
35. Karban, R., et al., Creating system engineering products with executable models in a model-based engineering environment. *SPIE Astronomical Telescopes + Instrumentation*. Vol. 9911. 2016: SPIE.
36. DOD, Department of Defense Digital Engineering Strategy. 2018, Office of the Deputy Assistant Secretary of Defense for Systems Engineering: Washington, DC, USA.
37. Defensa, M.d., Plan de Acción del Ministerio de Defensa para la Transformación Digital. 2020.
38. Ross, A.M. and D.E. Hastings, 11.4.3 The Tradespace Exploration Paradigm. *INCOSE International Symposium*, 2005. 15(1): p. 1706-1718.
39. Shallcross, N., et al., Set-based design: The state-of-practice and research opportunities. *Systems Engineering*, 2020. 23(5): p. 557-578.
40. Salado, A. and D. Selva, Asistentes cognitivos en ingeniería, in *UEM STEAM Essentials*. 2021.
41. Martin, A.V. and D. Selva, Explanation Approaches for the Daphne Virtual Assistant, in *AIAA Scitech 2020 Forum*. 2020.
42. Martin, A.V. and D. Selva, From Design Assistants to Design Peers: Turning Daphne into an AI Companion for Mission Designers, in *AIAA Scitech 2019 Forum*. 2020.
43. Martin, A.V.i. and D. Selva, Daphne: A Virtual Assistant for Designing Earth Observation Distributed Spacecraft Missions. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2020. 13: p. 30-48.

DR. DINESH VERMA

Dr. Dinesh Verma is the founding Executive Director of the Systems Engineering Research Center (SERC), and a Professor of Systems Engineering in the School of Systems and Enterprises (SSE) at Stevens Institute of Technology.



He was the founding Dean of SSE for 10 years from 2007 through 2016. During his fifteen years at Stevens he has successfully proposed research and academic programs exceeding \$175m in value. Verma served as Scientific Advisor to the Director of the Embedded Systems Institute in Eindhoven, The Netherlands from 2003 through 2008. Prior to this role, he served as Technical Director at Lockheed Martin Undersea Systems, in Manassas, Virginia, in the area of adapted systems and supportability engineering processes, methods and tools for complex system development.

Before joining Lockheed Martin, Verma worked as a Research Scientist at Virginia Tech and managed the University's Systems Engineering Design Laboratory. While at Virginia Tech and afterwards, Verma continues to serve numerous companies in a consulting capacity. He served as an Invited Lecturer from 1995 through 2000 at the University of Exeter, United Kingdom.

His professional and research activities emphasize systems engineering and design with a focus on conceptual design evaluation, preliminary design and system architecture, design decision-making, life cycle costing, and supportability engineering. In addition to his publications, Verma has received three patents in the areas of life-cycle costing and fuzzy logic techniques for evaluating design concepts.

Dr. Verma has authored over 100 technical papers, book reviews, technical monographs, and co-authored three textbooks. He received the MS and the PhD in Industrial and Systems Engineering from Virginia Tech, and was honored with an Honorary Doctorate Degree (Honoris Causa) in Technology and Design from Linnaeus University (Sweden) in January 2007; and with an Honorary Master of Engineering Degree (Honoris Causa) from Stevens Institute of Technology in September 2008. He is a Fellow of the International Council on Systems Engineering (INCOSE).

DR. ALEJANDRO SALADO

Dr. Alejandro Salado is an associate professor of systems engineering with the Department of Systems and Industrial Engineering and the director of systems engineering programs at the University of Arizona. In addition, he provides part-time consulting in areas related to enterprise transformation, cultural change of technical teams, systems engineering, and engineering strategy.



Alejandro conducts research in problem formulation, design of verification and validation strategies, model-based systems engineering, and engineering education. Before joining academia, he held positions as systems engineer, chief architect, and chief systems engineer in manned and unmanned space systems of up to \$1B in development cost.

He has published over 150 technical papers, and his research has received federal funding from the National Science Foundation (NSF), the Naval Surface Warfare Command (NSWC), the Naval Air System Command (NAVAIR), and the Office of Naval Research (ONR), among others. He is a recipient of the NSF CAREER Award, the International Fulbright Science and Technology Award, and several best paper awards.

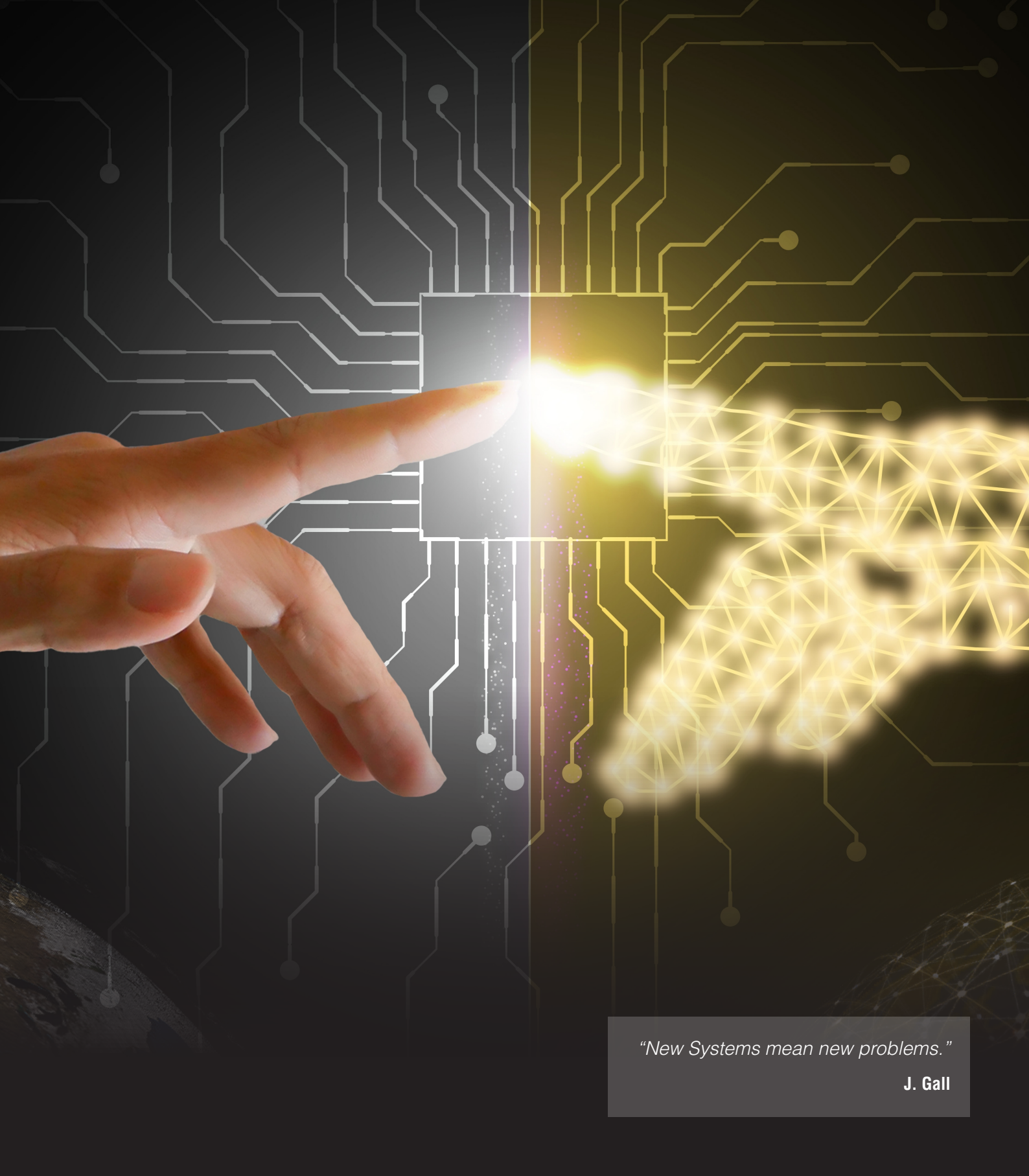
Dr. Salado holds a BS/MS in electrical and computer engineering from the Polytechnic University of Valencia, a MS in project management and a MS in electronics engineering from the Polytechnic University of Catalonia, the SpaceTech MEng in space systems engineering from the Technical University of Delft, and a PhD in systems engineering from the Stevens Institute of Technology.

ADOLFO SÁNCHEZ

Adolfo Sánchez holds a master's degree in Computer Science Engineering from the University of Zaragoza and a master's degree in History from the UNED.



He has participated in the definition and development of a modular avionics architectures evaluator, in the maintenance of the Eurofighter software, in quality assurance software developments made in strategic electronic warfare programs; in the acquisition, deployment, and implementation of the CIS Program of the Military Emergency Unit, and currently works supporting the Subdirector General of Digital Transformation of the Ministry of Defense and as an in-house trainer of various systems engineering courses in ISDEFE.



"New Systems mean new problems."

J. Gall

New Kinds of Systems

Tom McDermott, *Stevens Institute of Technology* (tmcdermo@stevens.edu)

Víctor Ramos, *Isdefe* (vramos@isdefe.es)

Abstract

This chapter presents four aspects of modern and future systems that are shifting traditional systems engineering practices: adaptability in systems, highly interconnected cyber-physical systems, learning-based systems with human-machine teaming, and distributed governance in systems of systems. Emphasis is given to current practices with doubtful effectiveness for such kinds of systems, then the chapter presents current trends on how to address these unique aspects of these new systems.

Keywords

Cyber-Physical Systems; Artificial Intelligence; Adaptive Systems; Agile Development; Learning Based Systems; Distributed Governance; Complexity.



1. INTRODUCTION

There are several trends that are changing the nature of the systems we develop and interact with today:

1. The pace of technological innovation continues to increase. With the availability of basic research on the internet, science and technology have become more of a commodity and the introduction of technical innovations into existing missions and platforms drives us to continuous adaptation and change. Systems are much less stable than they used to be, and competition favors those who can adapt the quickest to technology-driven change, not necessarily those who can invest the most to bring technologies to the mission.
2. The behavior of systems is and will continue to be defined more by software than hardware. All systems are or will be largely digital and will be supported by connected digital infrastructures. Hardware systems remain important but trend further towards more general purpose, more affordable and distributed even in critical applications like defense.
3. Data is increasingly collected and available for analysis. Computer storage and processing power continues to grow exponentially. Modern systems and future systems more so, adapt to changing internal and external conditions, as informed by changing data. The way we develop systems ought to change to better harvest and act on this data, and to take advantage of data and models to improve efficiency in development and management of programs.
4. Everything is becoming interconnected, and complex global systems of systems abound. Increasingly cyber-physical systems (CPS) are connected to other systems to share data and other resources as larger systems of systems. This opens both new opportunities and new vulnerabilities in system capabilities. Systems engineering (SE) rigor is necessary to balance the openness of these systems with other concerns such as safety, security, privacy, assurance, and alike.
5. Automation and user customization make systems more efficient, configurable, and adaptive. In particular, human tasks are much more dependent on teaming with automated or partially automated systems. This will continue to advance with artificial intelligence (AI) and machine learning (ML) to a new class of system known as a Learning Based Systems (LBS). An LBS is a new class of computing systems that achieves its function and performance through ML techniques. In the near future, human operators and LBS will both interact and adapt to jointly complete complex missions, a concept known as Human-Machine Teaming (HMT).

These trends are creating new kinds of systems, which we have categorized into four evolutionary and interrelated types:

- **Highly Adaptable Systems:** Modern systems make extensive use of software for their user-valued functionality as it is easier to change and minimizes production costs. Systems are becoming more and more tailorable in function to individual users. As a result, there is a trend toward systems that are easier to adapt to changing needs. In addition, highly adaptable development and support practices have permeated system business models to emphasize more frequent and consistent delivery of value in terms of capability introduction and flexibility. Learning and iteration have become more valued than up-front requirements understanding and stability, even in critical systems.
- **Highly interconnected Cyber-Physical Systems:** Modern systems are highly interconnected and fully dependent on software defined functionality. The Internet of Things (IoT) as an infrastructure is leading to large scale CPS systems such as smart cities, interconnected transportation, ubiquitous sensing, and manufacturing 4.0. Highly interconnected CPSs raise concerns over safety, security, and trust in domains where these have not previously been priorities. Large scale interconnected CPS increase the complexity and effective surface area of the systems we interact with. Digital twins (digital models of CPS connected to and operating with physical CPS) are now a rapidly growing domain of SE.
- **Learning-based systems and human-machine teaming:** Emerging systems will have increasing composition with LBS, which are systems whose behaviors are learned instead of programmed. These will be teamed with human users to augment human intelligence and agency (HMT). The impact of these systems on human tasking and situational awareness will also be an emergent property of future systems. LBS are expanding definitions of trustworthiness in systems, from primarily dependability concerns to human concerns such as ethics and fairness. Uncertainty management as well as test, verification, and validation of LBS are emerging challenges for SE.
- **Distributed governance in systems of systems:** The performance and capabilities of a system depend on information and shared functions with external systems that cannot be fully controlled. This leads to complex emergent behaviors and different methods of governance. Systems of systems (SoS) methodologies have been established to help manage governance, but as with other historical SE practices these are not keeping up with the rapid adaptation of SoS and shared governance.



Figure 1. Characteristics of increasing complexity in systems. [12]
Copyright © 2021 by INCOSE

The underlying theme across all four kinds of systems is the rapid growth in system complexity. Figure 1 highlights the related complexity concerns being driven by these new kinds of systems [12].

SE has always been viewed as a methodology to help manage system complexity, but the types of systems and their related system concerns are shifting, and the established methods are due to be updated. That is the subject of this chapter.

The next section evaluates the effectiveness of historically established SE methods in dealing with these new kinds of systems and SE concerns. Following that, we discuss some of the trends that are driving changes in the ways we practice SE as a result of these new kinds of systems. These discussions are not derived from exhaustive literature review, but from ongoing research and research roadmapping in the Systems Engineering Research Center (SERC) at the Stevens Institute of Technology in Hoboken, New Jersey, USA.

2. EVALUATION OF THE EFFECTIVENESS OF TRADITIONAL SE PRACTICES APPLIED TO THESE NEW KINDS OF SYSTEMS

2.1. Highly adaptable systems

Adaptability in systems is defined as the ability of a system to adapt itself efficiently and quickly to changed circumstances. An adaptive system is therefore a system that is able to fit its behavior according to changes in its environment or in parts

of the system itself [28]. The Systems Engineering Body of Knowledge (SEBOK) further discusses system adaptability as the system's ability to satisfy externally driven mission and requirement changes with or without modification, as measured by some value indicator such as cost, time, or resources [29]. This definition implies that context-driven change is an intentional process. This does not effectively capture the nature of today's LBS, which are continuously updating themselves based on learned behaviors or outputs that are responsive to changing external context. For this kind of systems, it is no longer appropriate to segregate a system from its external context; both must be understood and modeled.

Highly adaptable systems are software-intensive, highly connected, and have extensive automation and user configuration capabilities. At their core are sets of data that drive the behavior of the system, which is defined by software logic, algorithms, and control and data management functions. The underlying mechanism is a connection between the sets of data in the system and the external environment that allows the behaviors of the system to rapidly change in response to external context changes. This can be an intentional data-driven learning and modification process responding to changing mission, or can be highly automated in response to changing external context as with emerging LBS.

However, SE, strongly influenced by aerospace and defense needs, has continued to be linked with the physical realization of large complex systems and other critical capabilities that are intended to persist for many years. The need for rigorous definition, analysis, and testing of these critical systems will always exist, but the lifecycle processes we choose to use

should be tailored to the system’s actual use and life. For example, SE for modern systems tends to be more model-based, agile, and responsive to user needs, which may be accomplished with more adaptable and efficient lifecycle processes by leveraging data and models [16]. Software systems engineering (SSE), information technology and enterprise architecture, distributed modeling and simulation, and automated manufacturing systems must all be leveraged in a convergent fashion to address lifecycle management of highly adaptable systems.

Modern SE technical and management processes transform data into views through models, which support analyses leading to decisions. This digital process flow supports “Data Transformed into Models then Analyzed through Views to make Decisions documented in Digital Artifacts.” This process flow is not new, but is evolving from a largely manual, inefficient process flow to a highly automated process flow driven by rapid, consistent, and value-driven adaptation cycles. (Software development practices have evolved to manage this automation.) Figure 2 redraws the widely depicted “Define -> Realize -> Deploy&Use” stages of the SE Vee-model lifecycle process in a circular process to represent it as a:

- 1) set of data at the core, interpreted by model transformations, leading to design decisions,
- 2) layered across disciplines and engineering tasks to produce decision artifacts, and
- 3) in continuous iterative processes that could be entered from any point.

The challenge in highly adaptable systems is maintaining appropriate SE rigor and associated process definition to ensure these systems remain scalable, resilient, safe, secure, and usable by human operators. New SE lifecycle processes address shared and authoritatively managed sets of digital data and models associated with the system’s entire lifecycle, not just a single engineering or program lifecycle [16].

2.2. Highly connected Cyber-Physical Systems (CPS)

Highly adaptable systems are evolving from software-only systems to fully connected software/hardware systems known as CPS. The U.S. National Science Foundation (NSF) defines CPS as “engineered systems that are built from, and depend upon, the seamless integration of computational algorithms and physical components” [19]. A CPS has computers and networks that control physical processes, often characterized by feedback loops that affect computations and the physical outcomes of those computations. Figure 3 provides a general layered depiction of a CPS framework [11]. As shown in the figure, the design of CPS must address these control activities in a device of interest but also the interconnected human and machine systems that interact with it, which may occur at long ranges over cyber networks [10].

Furthermore, CPS transform the way in which people interact with systems [19]. For example, humans can now interact with engineered systems over cyber networks instead of directly (such as controlling a thermostat from a mobile phone) or even across multiple interconnected CPS and large-scale software

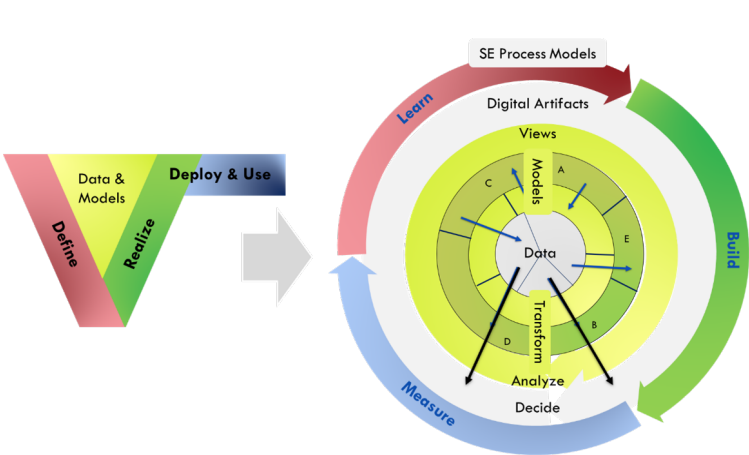


Figure 2. Circular Processes with Data at the Core [16]

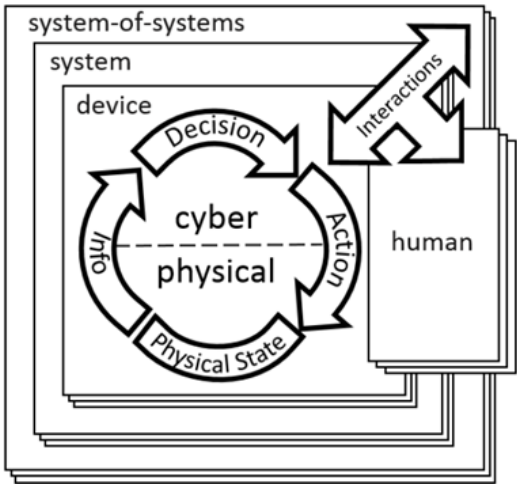


Figure 3. A general framework for CPS [11]. Republished courtesy of the National Institute of Standards and Technology

systems. These interactions are enabled by higher degrees of automation and autonomy [19], which increase however the complexity of the system control methods and the speed of evolution of the system in response to its external domain.

Traditionally, SE has distinguished how a system is constituted internally (i.e., its structure) from how the system manifests itself externally (i.e., its behavior) and deduced the function from the structure. This notion is the foundation underlying hierarchical construction of systems with defined input and output interfaces across multiple modular components. It has resulted in a focus on structural system representations in SE, supporting physical trades in primarily physical systems. Highly connected CPS, however, are heterarchical in nature. They are comprised of numerous, heterogeneous elements acting both independently and interdependently. Because of this complexity, traditional structural decomposition and physics-based models are insufficient. In complex systems, form (structure) and function (behavior) are intrinsically linked and not separable. The complexity of the system control methods (behavioral) and the evolution of the external domain that interacts with the system (adaptive) cannot be ignored. These changes are a product of the computer/network interactions and increased use of digital data in the control functions, along with the connected nature of the external environment and users.

As an example, Figure 4 shows a functional and structural view of the highly cyber-physical systems emerging in transportation systems today. In practice these systems

are more often built from general purpose programmable hardware while behaviors are programmed by software. Their function in the full system may evolve incrementally over time, as we are seeing in vehicle automation today. Mechanisms that determine the qualities of these system, such as safety, adaptability, or resilience, are intentionally supported in the hardware and software designs but also realized by investment in both system structure/function today and architecture evolution over time. This is a concept known as technical debt, where decisions made in the current design gradually limit the system's ability to support new capabilities in the future. Informing decisions that drive both the immediate and long-term qualities of the systems they support are thus critical.

The complexity of these networks of CPS results in emergent behaviors that cannot be fully modeled and predicted, or even structurally decomposed in traditional ways. The technology traditionally employed by the SE is also going through an evolution towards digital engineering (DE) and simulation-based design practices. Time invested in up-front digital simulation of these emergent behaviors is essential to both short-term and long-term design decisions. These simulations can also be deployed to monitor the CPS behaviors after installation to more rapidly and accurately detect undesired behaviors in use. This is the driver for and concept behind digital twins, which will be expanded in Chapter 6. In this paradigm, both the realized system and the virtual simulated system and interconnected products are holistically planned and managed with full SE life cycles. As

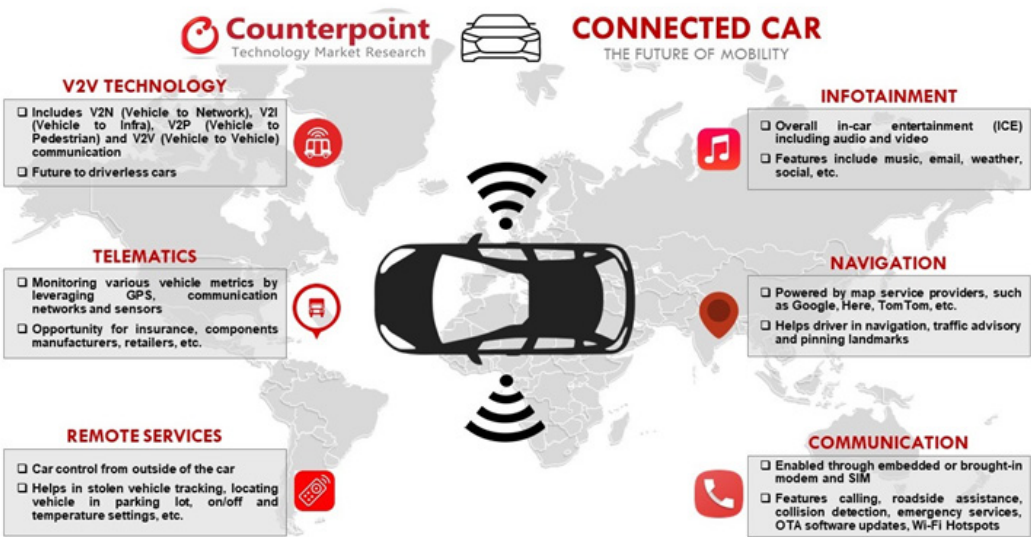


Figure 4. The automobile as a highly connected CPS [2].
Image from <https://telecom.economictimes.indiatimes.com/tele-talk/connected-car-opportunity-propels-multi-billion-dollar-turf-war/2971>

can be seen, the concerns of a modern systems engineer expand in this context from just the structure/behavior and domain-driven non-functional qualities of the system itself, to also include its interaction in the larger connected context and environment, and finally to how both the realized system and its companion virtual twins are managed and evolved over the full lifecycle. This has become a multi-disciplinary challenge requiring a shift in SE skillsets to integrate across the principles, foundations, and characteristics listed in Table 1 in a holistic manner.

The SE concerns of CPS can also be stated in terms of trustworthiness, related to the ability of the CPS to withstand instability, unexpected conditions, and gracefully return to predictable but possibly degraded performance [11]. This is truly a system concern in highly connected CPS. These characteristics of trustworthiness, which include dependability, safety, reliability, privacy, security, and resilience, have for the most part evolved within distinct disciplinary and education silos. Historically, SE practice has treated these as disparate sub-disciplines. Large SE and integration projects often have property-specific leads, who

represent discrete viewpoints within the trade-off process overseen by the chief systems engineer/integrator. Functional requirements often have caused engineers and designers to prioritize each property differently, based on domain-specific requirements and perspectives (e.g., energy, manufacturing, transportation, etc.). Achieving a certain level of success in each property is typically vital to the overall success of the system. Trends in the engineering of highly connected CPS suggest that SE disciplines are converging toward increased interdependency.

This is particularly important for highly connected CPS, in which systems-based holistic thinking is critical to supporting trustworthiness objectives and avoid problems arising with respect to one property, or protections inserted to address one dimension of concern, do not compromise other primary system objectives or cause deleterious unintended effects [17].

Awareness	Supervised practitioner	Practitioner
Communication and Networking	Basic computing concepts, including software engineering	Security and privacy
Embedded systems, both hardware and software	Discrete and continuous mathematics	Discrete and continuous mathematics
Real time systems	Physical world computing, including sensors, actuators, and real-time control	Interoperability
Physical world computing, safety, reliability, security, performance, and risk management	Cross-cutting application of sensing, actuation, control, communication, and computing	Reliability and dependability, Safety, Stability and performance
Human interaction with CPS, including ease of use	Modeling of heterogeneous and dynamic systems integrating control, computing, and communication	Human factors and usability
	CPS system development (emphasizing concepts of resilience and safety, test and verification)	Power and energy management

Table 1. NAS CPS principles, Characteristics, and Foundations [18]

Thus, we have today a multi-disciplinary challenge associating together foundational disciplines of various schools, such as engineering, computing, and human interaction. We also have a multi-disciplinary challenge across traditional SE sub-disciplines related to (1) dependability in computing systems (availability, reliability, safety, integrity, and maintainability), (2) system security engineering (confidentiality, integrity, and availability), (3) information systems (management, communications, and privacy), and (4) cybersecurity (threats and protection). This is also a system governance challenge, both in the enterprise systems that develop highly connected CPS, and in the emergent behaviors of the CPS themselves [17]. There is a need for a more interdisciplinary approach to system design, founded on rigorous system functional modeling and simulation, evolutionary design, and rigorous evaluation. As digital engineering and Model-Based Systems Engineering (MBSE) become more prevalent, there is potential to transform traditional system design and evaluation processes to more holistic and more evidence-based forms using models (to be further elaborated in Chapters 4 through 6). Verification and validation of highly connected CPS through test and evaluation have historically been the gold standard but is significantly expensive and fraught with difficulty as systems become more complex, more expansive, and more inter-dependent on other systems to realize their intended capabilities. Again, bringing data and models into this process aims to relieve some of the expense and make the entire process more flexible and amenable to changes that can occur across a system's lifecycle.

2.3. Learning-based systems and human-machine teaming

Concepts of highly adapted systems, highly interconnected systems, and resulting distributed governance are placing SE in the midst of a digital transformation driven by advanced modeling tools, data integration, and the resulting digital twins. Data, models, and computing systems are all converging with machine learning (ML) techniques to enable a new class of computing system that achieves its function and performance through the use of ML – known as a Learning Based System. Applications of LBS are evolving exponentially into many domains.

LBS are types of AI models that use ML algorithms to make decisions and solve problems without being explicitly programmed. ML algorithms independently detect, analyze, and learn patterns directly from input data and modify their behavior accordingly to predict new output. LBSs differ from

a longer legacy of rule-based AI systems in their ability to achieve scale, provide adaptability, and handle complex tasks. Rule-based systems are a type of AI model that uses a set of prewritten rules to make decisions and solve problems. Developers create rules based on human expert knowledge that enable the system to process input data and produce a result. Almost all systems of the future, and the tools we use to design them with, are evolving to include compositions of rule-based and learning-based components.

The number of systems with some level of learning capacity is increasing exponentially today. In some applications, these systems work closely with humans; in others, operations are largely autonomous. In these systems, distributed teams of humans work with distributed teams of autonomous systems, with relationships that can change dynamically. Likewise, SE is evolving to more of a technological discipline that uses LBS to define and manage digital data and descriptive models that link different disciplines together. The SE digital engineering transformation is being followed by transformational advances in the discipline of SE using AI and ML technology for automation of many engineering tasks, designed to augment human intelligence. At the same time, the application of AI, ML, and autonomy to many of today's complex and critical systems drives the need for new SE methods, processes, and tools.

A primary goal of SE is to ensure that the behavior and performance of complex engineered systems meet the expected outcomes driven by user needs, and that the configuration of the system is managed across its lifetime. Advances in AI and ML application means that future system components may learn and adapt more rapidly, and that behavior and performance may be non-deterministic with less predictable but manageable outcomes. This is the LBS challenge. The inability to explicitly validate system behaviors or the time it takes to do that will impact trust in these systems and is driving change in the way the SE community traditionally addresses system validation [26]. The uncertainty present in multiple AI/ML components that interact defies traditional decomposition methods used by the SE community, requiring new synthesis methods. Finally, as systems develop means for co-learning between human users and machines, traditional models that separate human behaviors from the machine will need to be revisited [14].

Thus, the emerging SE challenge is to produce systems that gain value from their ability to learn, which may be inherently non-deterministic, but that are also appropriately robust, predictable, and trustworthy in the type of critical and complex uses common to the application of SE practices

today. This includes both human and machine behaviors in joint decision environments, highly reliant on good human-systems design and presentation of decision information. It also includes the adaptation of test and evaluation processes to co-learning environments [14].

The future SE challenge involves LBS that actually adapt and learn dynamically from their environments. These environments could be real, simulated, or a mix of both thanks to the generation of synthetic data, which offers new approaches for system training and development but, on the other hand, generates the need for synthetic data validation. In this rapidly emerging future, machine-to-machine and human-to-machine (and maybe machine-to-human) trust will be critical. In this future, systems will be expected to learn to modify or create new behaviors as the context changes and this may happen fairly rapidly. Methods that revalidate system performance extremely rapidly or “on the fly” are not part of the current SE practice set and must be developed along with these types of learning systems [14].

The future of LBS and SE is difficult to predict at this point. The SE discipline is entering a period of rapid change where it will be trying to catch up to the evolving use of ML algorithms and resulting LBS. The complexity of engineered systems is rapidly increasing, but the automated tools that systems engineer’s use are also evolving to manage that complexity – albeit at a slower pace. The transformation of SE away from manual paper-based workflows to a fully digital and model-based set of practices is actually quite urgent.

2.4. Distributed governance in systems of systems

The distinguishing feature of a system of systems (SoS) is the behaviors of the “whole” come from individual constituent systems that act independently and autonomously [3]. Furthermore, SoS are generally sociotechnical systems: technology-driven systems that involve significant human and social participation, where that participation in turn influences the architecture and design of the technical system [13]. In other words, the human is a part of the system, not an external agent.

In order to determine appropriate architecting principles, SoS engineering (SoSE) literature defines a classification system for SoS linked to the degree of managerial control in the SoS. The four SoS classes are Directed, Collaborative, Virtual, and Acknowledged SoS. The degree of central control or governance over SoS changes is the primary distinction of each class [4]. In a directed SoS, the integrated

SoS is created to serve a specific purpose and is governed centrally. The constituent systems remain independent, but their normal operations are directed through a central authority. In a collaborative SoS, the constituent systems volunteer to collaborate and there is not a central control agent with authority to direct their operations. However, there is still a governance function determining an agreed upon purpose for the SoS. In collaborative SoS standards, regulations, norms, and circumstances drive the operation of the constituent systems.

SoS literature additionally distinguishes the collaborative SoS classifications as virtual and acknowledged. Virtual SoS are collaborative but develop without a centrally agreed upon purpose or set of goals. Governance is fully distributed. Acknowledged SoS have a recognized objective set and a designated managerial component, have resources allocated for development, but changes are based on collaboration objectives. Governance remains distributed but conforms to centrally determined policies and outcomes. A good example of a collaborative SoS is automotive traffic on the interstate highways. Whether or not this highway traffic SoS is viewed as virtual or acknowledged is linked to stakeholder perspectives. Drivers would view it as virtual (its goals are determined by my individual need to get from point a to point b), while highway transportation officials would view it as acknowledged (its goals are to safely and efficiently manage interstate traffic). Today’s and future adaptable and highly interconnected systems are more collaborative and struggle with managerial control, and individual perspectives and behaviors will affect system behaviors in ways that cannot be easily predicted by traditional SE decomposition.

The Wave model is an established framework for evaluating and planning evolution in systems of systems (ref. Figure 5). The model recognizes that evolution is continuously driven

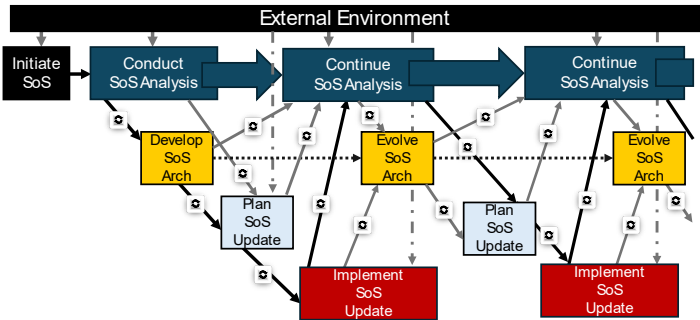


Figure 5. The wave model [7]

by input from the external context, and unlike traditional SE it views the analysis of system change as an ongoing process with multiple overlapping increments. In the Wave model, system evolution is a forward-looking process with feedback at each iteration, and managerial control strategies attempt to group multiple constituent changes into SoS level architectural changes to create efficiency in the test and validation process [7].

A different perspective is given in the innovation literature, called “transition management” [21]. Innovation literature counters the Wave model with a more bottom-up view of system evolution. Innovation system models recognize innovation as a complex adaptive process where lower-level innovations in constituent systems form niches of adoption, which over time produce broader changes in established SoS regimes, eventually resulting in transformation of the existing landscape (or context). Today one can view the public development of driverless automotive technologies as such an evolution “in-process.” The primary aspect of this model is that innovation progresses through social layers and can be modeled as a

multi-scale or multi-layer social phenomenon, as opposed to the more mechanistic view of the Wave model. Figure 6 shows this process as reflected in transition management literature.

The competing phenomena in these two views of SoS evolution are differing models of distributed governance. The SE perspective in the wave model assumes that SoS change can be planned and managed over time by some governance mechanism. The innovation perspective in the technology transition model notes that change will evolve from bottom-up technology evolution that cannot be governed but can be encouraged or guided. In defense systems, distributed governance of the SoS and constituent systems is a set of agreement processes between military organizations. In commercial innovation, distributed governance is determined by market factors. A SE challenge today in this regard derives from the commercial marketplace driving technology innovation and the defense and aerospace industries primarily driving SE methods and tools in a somewhat insulated form from that larger marketplace.

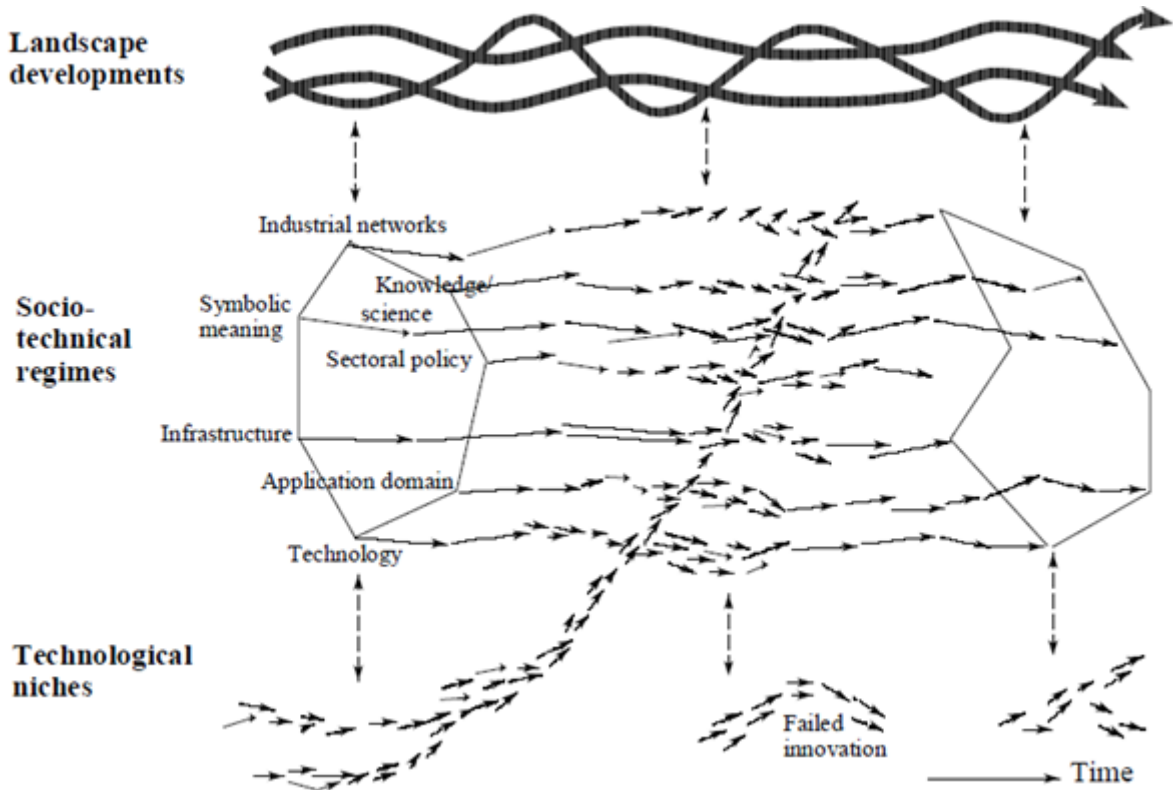


Figure 6. A dynamic multilevel SoS perspective on technology transitions [8]

This brings the concern of current SE methods being slow to change and not adapting well to the current dynamics of distributed governance in more adaptable, highly interconnected systems. Trying to fully predict emerging SoS requirements and rigidly plan SoS updates up-front runs counter to current commercial business markets. Commercial businesses do aim to centralize governance of SoS value but recognize actual SoS emergent capabilities require openness and experimentation ahead of rigid planning.

Technology is also driving the execution of distributed governance models. Recent technologies such as software orchestration and blockchain/distributed ledger technology (DLT) have led to the emergence of “leaderless organizations” where rules and system goals are distributed via software automation and people are allowed to self-organize their tasking. These same technologies will likely help to manage “authoritative data and models” in future SE, but the governance of these is an open question at this point. This is not just a commercial trend; emerging military command and control concepts envision future military SoS as an internet of things, ideally moving from rigidly connected platforms to more flexible distributions of capabilities managed by distributed control centers in line with the concept of collaborative combat. The same concepts of highly adaptive systems, highly interconnected CPS, and LBS/HMT emerge together to transform military operations. All these trends point to the need for SE methods and processes that center on agility and flexibility, along with improved integration of the business reasons for these traits.

3. TRENDS TO EVOLVE SYSTEMS ENGINEERING TO EFFECTIVELY ENGINEER THOSE TYPES OF SYSTEMS

3.1. Digitalization

The core change driving the discipline of SE today is the transition towards digital engineering, a paradigm where “shared and authoritatively managed data” can be transformed through “shared and authoritatively managed models” and related tools to create Digital Artifacts that can be used by various decision-makers and others needing digital access to the design and descriptions of the system across its lifetime [20]. These artifacts were almost always paper documents or drawings in the early years. Now they are generally based on digital technologies, but workflows still tend to be document driven instead of data driven.

The workflow view in Figure 7 shows conceptually how shared and authoritatively managed data should be transformed into digital artifacts in different life cycle stages in any SE process. Data, federated data models, and distributed data storage are the foundational infrastructure of modern SE. This is a distributed federation of data and models, governed across organizations, with authoritative processes to manage data and model provenance and configuration.

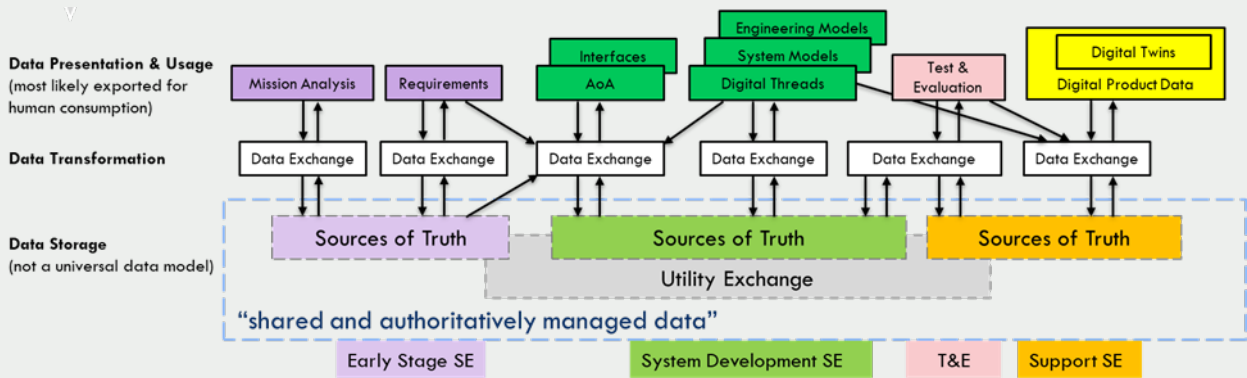


Figure 7. Data Transformation into the Life Cycle

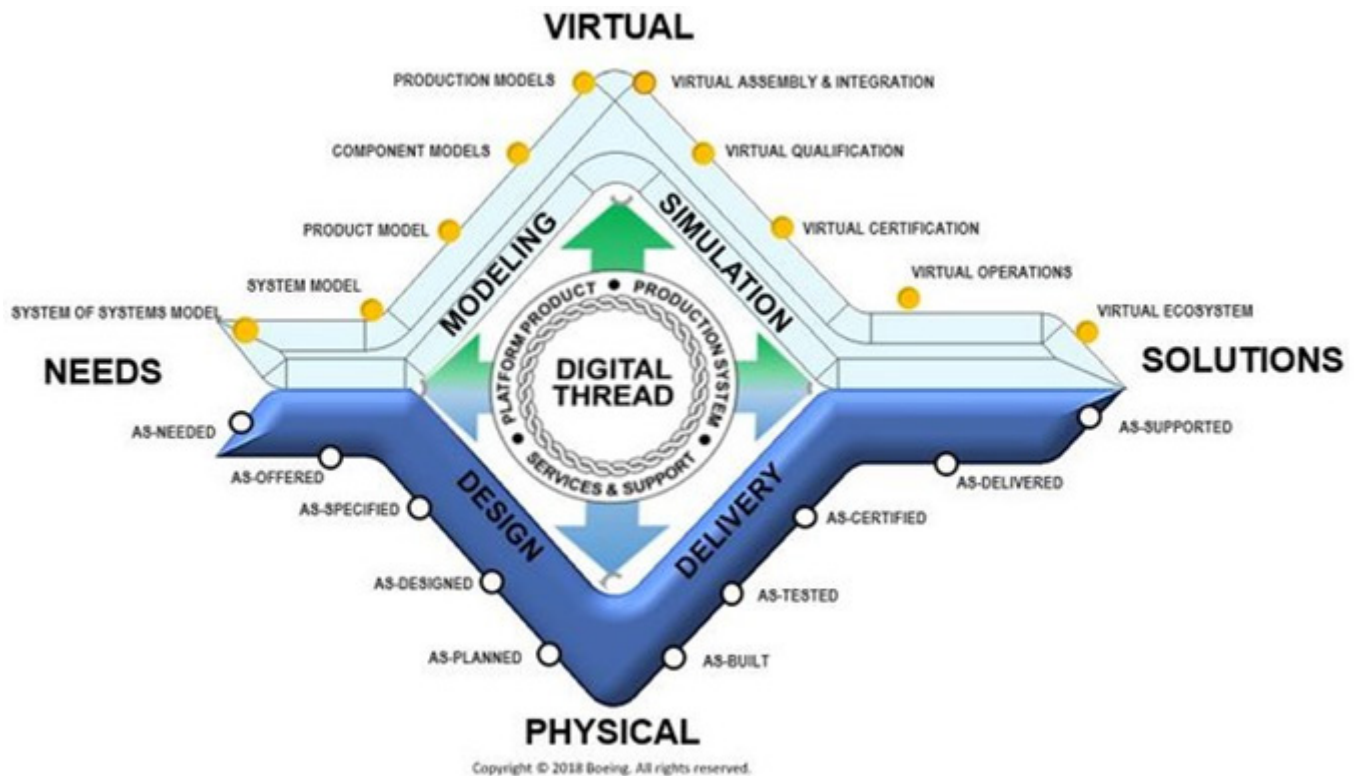


Figure 8. The Boeing “Diamond” showing the full lifecycle existence of virtual and physical (adapted from [5])

Figure 7 is particularly relevant to SE modernization, as “Data Management” is not currently defined as a disciplinary process in SE standards. Data, models, and data storage systems can each be conceived and treated as a separate system that must also be developed and deployed in support of the fielded system. These must be defined and built along with other system development aspects. These also have their own lifecycles. New SE lifecycle processes are evolving to address shared and authoritatively managed sets of digital data and models associated with the system’s entire lifecycle, not just individual engineering or program lifecycles. This mental model has been depicted as a combination of a conventional Vee model with a mirrored Vee model to create a design diamond that incorporates digital counterparts of a product at all stages of its lifecycle (ref. Figure 8) [5]. The virtual system and the physical system are anticipated to evolve together in the long-term across the full system lifecycle.

Systems engineers have long used digital data and various modeling and analysis tools to produce digital artifacts for decision-making (such as Microsoft PowerPoint slides).

However, the underlying data models have not been “seamlessly shared” in digital workflows and tools, or likely not shared at all. Furthermore, authority for that data has often been held by independent activities generally organized by discipline. Today, much of the transformation from data to models to decisions is still a manual interpretation of disparate data and analyses. This manual interpretation limits our ability to be iterative and responsive across disciplines and disciplinary tools. It is inefficient and non-holistic. The evolution of SE is a fully integrated, iterative workflow where the system is the focus, not the owner of the data or the particular element of a design. Today’s primary challenge in digital engineering is not so much being “model-based,” it is understanding, creating, and validating the underlying data model that integrates across requirements, design, test, disciplines, and disciplinary processes, with it being shareable and shared. The value of SE in the future can be realized through a more seamless and efficient transfer of data and models, starting from underlying performance drivers through models to decisions and ease of drilling back down from decisions to data. This is currently not the state of SE practice, but a target to define capability roadmaps.

3.2. The digital ecosystem

Collaboration between disciplines and organizations requires establishing high assurance interfaces between multiple engineering, development, and management applications in a digital engineering ecosystem. The future digital ecosystem for SE is envisioned as a service-oriented architecture to provide flexibility and adaptability between tool suites across and between organizations, as shown in Figure 8. Data exchange between applications may be technically envisioned as a set of collaborative APIs for data and information sharing, as well as query and response. Maturing standards, moving towards open systems, and building up experience implementing digital SE are necessary to facilitate the adoption of digital engineering in organizations [16].

3.3. The digital system model

One of the most impactful changes in SE in recent years has been the maturation and adoption of MBSE, which is presented with certain level of detail in Chapter 4. MBSE is the

formalized application of modeling to support SE activities, including system requirements, design, analysis, verification, and validation activities beginning in the conceptual design phase and continuing throughout development, production, and later life cycle phases. MBSE has a particular value in the digital ecosystem as an approach to express and capture the relationships, interdependencies, and associated processes connecting systems level models and other disciplinary models as well as the life cycle process flow. For example, system models are useful for showing relationships among requirements, system functions, physical components, suppliers, acquirers, and users. Being machine-readable, formal models allow relationships between data elements to be established and to be processed by software thus enabling efficiency-improving capabilities.

In MBSE, the system model is used as the central repository for design decisions that span multiple engineering and business concerns; design decisions are captured as model elements in that digital system model [6]. This is the emerging digital product of the SE function.

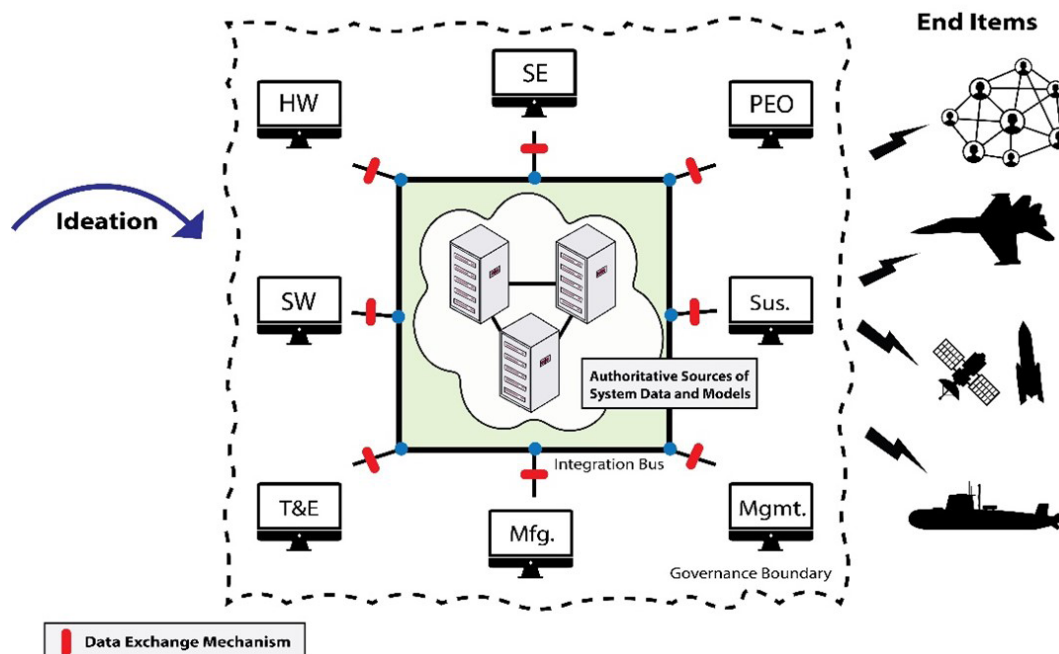


Figure 9. An Operational View of a Digital Ecosystem.
(SE: systems engineering, HW: hardware, SW: software, T&E: test & evaluation, Mfg: manufacturing, Mgmt: management, Sus: sustainment, PEO: program executives)

The practice of SE is still learning the most productive uses of MBSE. Initially the focus has been on a digital representation of the system specification, to include concept of operations, requirements, work breakdown structure, use cases and functions, and performance analyses. The concept of the digital system model as the central repository for design decisions (or at least an index to this) informed by executable simulations and/or quantitative analyses is the core use of MBSE in DE, and is still evolving, together with systems modeling languages and tools.

The use of a digital system model should eventually apply to all systems in all domains, but the concept is still in its infancy. Most systems are still developed and deployed without this formal but holistic set of models and views, leading to continued late lifecycle failures and rework. As new kinds of systems become more adaptable, interconnected, distributed, and learning-based, the importance of the MBSE activity should gain value. While there is still a way to go in terms of maturity of capabilities enabled by MBSE, initiating the transition is already feasible and valuable.

3.4. Data models, ontologies, and semantic web technology

As the digital basis for SE grows, it is relying more on defined and structured data (data modeling). SE practices now include machine usable languages and development of taxonomies, lexicon, and data ontologies to enable interoperability for computationally based data synthesis, analysis, and exploration. Connectivity across the digital system model starts at the data layer. Development and maintenance of an enterprise data model and environment has become part of SE work. The data model defines the specific structure and relationships of the data elements that inform higher level system models and analyses. A digital ontology is a reusable framework that models generalized data – general objects that have common properties and not specified individual entities. A data model that rests on top of a digital ontology supports greater interoperability and reusability of the data.

A recent SERC workshop report on digital ontologies summarized the need for and value of effective digital ontology development as one of the means to digital interoperability [25]. Traditional SE applications mainly rely on documents to capture and exchange information. Documents do not promote interoperability, efficiency, and automation, given their lack of formalism in both syntax and semantics. Ontology-enabled methods allow us to make inferences on

data and information to determine new facts and discover previously unseen gaps and relationships. They also make it possible to uniquely identify data elements so that different data systems can refer to the same concepts without having to pass around or duplicate unwieldy data structures. These ontologies are a pragmatic means to formally describe the relevant entities and relations of a system, implement those descriptions and relationships, and be useful for our specific purposes and objectives. For example, the Common Core Military Ontologies represent data from different military branches in a common structure that allows data sharing and data aggregation in each domain using generic reusable data representations. The common ontology enables data interoperability in joint operations. In SE, syntactic interoperability across formal requirements, design, and test language is vital, but insufficient. Data interoperability is necessary to enable the digital infrastructure to exchange and aggregate requirements, design, and analysis information. The value of ontologies lies in what they allow us to communicate, and multi-disciplinary communication is at the core of SE and DE.

The SE community needs but does not yet have a broad understanding of the different ontological and related data modeling methods necessary to ensure data sharing and interoperability. The SE community is set to expand education and training to include those foundations of data architecting, data modeling, and data science specifically relevant for tomorrow's systems engineers.

3.5. Agility

Data transformations into and out of the shared and authoritatively managed federations of data and models described in the previous section are expected to happen iteratively and continuously across the entire life of a system. These data and models might originate in any phase of a system's lifecycle and any function associated with engineering and management. To respond to this new context, SE practices are becoming more agile and responsive. Increasing responsiveness does not mean eliminating critical SE processes, just increasing the number of iterations and shortening the cycle time between them. A recent workshop on agile practice in hardware-intensive systems captured a number of themes that must be better adopted into SE practice [1].

Commercial industry has adopted agile practices in software, hardware/software systems, and services to address rapidly changing threats to and opportunities in their business. A

primary goal of agile practices in SE is to shift learning to as early as possible in the development process, that is, being intentional in the early design stages of systems to accommodate innovation in later stages of development. Agile practices augmented by digital models, prototypes, and test infrastructures help bring learning forward, reduce integration risks, and create more flexibility in long-term design decision points as a result. Choosing the appropriate systems, subsystems, or elements of the system to emphasize in this strategy helps anticipate evolution of parts of the system that have the most potential for change or those for which innovative change will have the most pronounced performance gain. Newer areas critical to SE, like software SE, information technology, enterprise architecture, distributed modeling & simulation, and automated manufacturing systems enable this transition.

The underlying premises of agile practices include direct collaboration between users and developers, encouragement of simplicity, and creation of continuous flow of value. In large scale defense systems, for example, the flow from warfighter need to capability passes through many organizations and

processes before it becomes a program (of any type). This changes interpretation of needs and requirements, isolates the real customer from the capability development, and interrupts the flow of work from need to capability. On the contrary, commercial industry has pioneered many different ways to determine customer needs and responses using analytics embedded in the products themselves, accelerating customer understanding.

Similarly, agile practices challenge the single batch mindset and the belief that everything must be understood before implementation. This implies an enterprise-level shift to allow more frequent delivery of working systems (or system elements) through reconciliation of development and delivery cycles for best effect. Rather than compounding the effect of slower cycles that drive the pace of system-level delivery, a refactoring of the contributing streams of work can assure flow enabled by smaller batches of work. Milestone completion remains important but must be translated into buying down risk, not just criteria completion. Integrating both a consistent work cadence and milestone-driven goals are critical to agile in hardware-intensive systems. Meaningful movement of

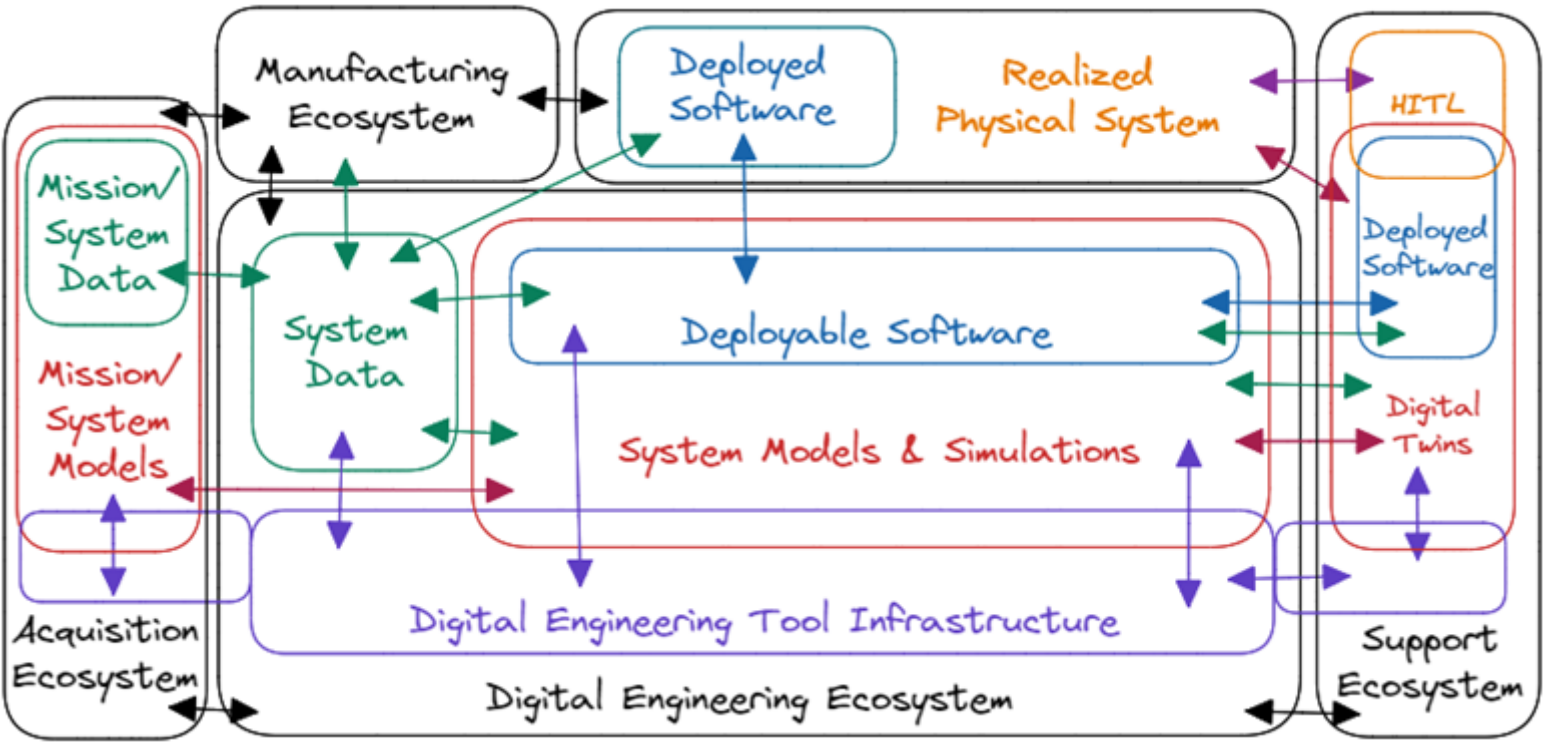


Figure 10. Today's systems should plan for deploying working software frequently into all aspects of program planning and development

prototypes from virtual environments to physical realizations to operational use has tangible benefits when the software is reused from one product to another. Programs should embed deployable software into simulation and training systems, allowing all developers and users to experience the operational use of the product and enabling Live Virtual Constructive (LVC) environments. Figure 10 depicts this approach.

What remains consistent with current SE practice is thoughtful decomposition and partitioning. Breaking down complex systems into smaller, manageable components allows for faster learning and better understanding of individual elements. Agile practices take advantage of modularity to architect systems that can be evolved over time. Control of interfaces and application program interfaces (APIs) is fundamental to both defining the work in the system and the team skills needed to do the work. Modular Open Systems Approaches (MOSA) precede agile development in both software and hardware systems. As a note, this partitioning must consider existing structural system decompositions as an area for innovation as blindly following traditional subsystem decompositions can limit agility.

Finally, agile practice in hardware-intensive systems requires front-end investment in test activities and infrastructure to buy down end-item risk. For example, SpaceX™ considers launch failures a data collection investment. Their mindset of corporate learning from multiple launch failures is a strong example of the culture and mindset required for innovation and continuous improvement in agile practice [9]. Investment in model-based engineering tools, multiple systems-level prototypes, and hardware-in-the-loop environments is critical for successful agile implementation in hardware-intensive systems. This is more difficult in hardware-intensive programs than software-only programs because the tools are more diverse and less well-integrated than in today's software development environments.

3.6. Modular and Open Systems Approaches (MOSA)

In highly adaptive and interconnected systems, as mentioned earlier, one needs intentionality in the early design stages to accommodate adaptation in later stages of development. Particularly in large scale critical systems like defense and aerospace systems, platforms may have lifecycles spanning decades. Choosing the appropriate systems, subsystems, or elements of the system to emphasize in this strategy helps anticipate evolution of parts of the system that have

the most potential for change or those for which innovative change will have the most pronounced performance gain. Choosing the related business strategy is equally important to the success of the system over its life cycle. This is the driver behind a MOSA – an intentional design approach to maintain a technical and business basis for future flexibility and innovation in the system. Decomposition into smaller and smaller functional capabilities and hardware components reduces complexity at the component level, as well as the size of the associated work teams. However, modularity must be carefully designed and planned as an architectural strategy in a lifecycle planning activity. The skills to do this successfully are specialized and in high demand.

Decomposing to smaller modules increases agility but also increases integration risk. Standardization of interfaces, and particularly the use of open interfaces, can help reduce such risk. Open standards were almost always defined by documents in the past, making interpretation, compliance, and actual interoperability a challenge. With the introduction of the Internet Protocol (IP), “plug and play” capabilities into Microsoft Windows™ products, and the Android™ operating system, open standards have been more often implemented as open-source software. The adoption of digital engineering and a digital system model facilitates formalizing the interfaces between modules in a digital baseline to eliminate interpretation and miscommunication [22].

3.7. AI4SE and SE4AI

It is difficult to predict exactly how AI and ML will impact SE today, but in the near future SE will undergo significant change in methods, processes, tools, and skills as part of the AI and ML megatrends. At an early 2019 Future of Systems Engineering (FuSE) workshop hosted by INCOSE, the terms AI for SE and SE for AI were first used to describe this dual transformation [15]. The “AI4SE” and “SE4AI” labels have quickly become metaphors for an upcoming rapid evolutionary phase in the SE Community. AI4SE may be defined as the application of augmented intelligence and machine learning techniques to support the practice of SE. Goals in such applications include achieving scale in model construction, confidence in design space exploration, and automated validation coverage. SE4AI may be defined as the application of SE methods to the design and operation of learning-based systems. Key research application areas include the development of principles for learning-based systems design, models of life cycle evolution, and model curation methods [14]. These are summarized from published SERC roadmaps on AI and Autonomy drivers for SE [14].

SE and software engineering are fields that have language-based models that are very amenable to machine automation. Tools exist today that use large language models to author software code. AI-based tools that author SE requirements and descriptive models are beginning to emerge at the time of this writing. Some of the technologies and use cases to watch for include automated construction of models from features in semantic data, used in both creation of new models and correctness of developed models; automated search through data and models; automation of evidence-based models for assuring correctness and completeness of system requirements and design; automation of certification and accreditation processes via models and automation of quality assurance data; and eventually chatbots or cognitive assistants that automate many mundane data entry, exploration, engineering calculation tasks, and workflows.

Likewise, the emerging needs levied on SE by LBS will introduce significant changes in SE methods, processes, and tools. Most of these are reflected in the other trends in sections 3.1-3.6. Some of the more direct expectations include:

- Architecting new combinations of live and virtual digital architectures that dynamically adjust their structure and functionality.
- New methodologies to support human and machine situational awareness of impending risks and behavioral concerns.
- Methods, processes, and tools to connect system risk analysis results with AI software modules related to those risks.
- New analytical and evaluation methods that calibrate trust in LBS, beyond traditional availability and dependability concerns to concepts of ethics, fairness, etc.
- New mission level analysis and risk models arising from human and AI collaboration in shared tasks and functions.
- Methods for addressing AI-related system test and evaluation addressing these systems' ability to adapt and learn from changing deployment contexts.
- Computer-based simulation and training supporting non-static objectives and/or goals (games, course of action analysis) necessary to provide contextual learning environments for these systems.

This will also be a transformation of the SE workforce, with significantly more integration of software and human behavioral sciences at the forefront. As digital engineering evolves and traditional engineering models and practices rely more on the underlying data, many engineering tasks related to data collection and search, data manipulation, and data analysis will become automated. Also, the machine learning of modeled relationships and underlying data will become more powerful over time. This should be a positive change, automating many mundane engineering tasks leading to a greater focus on problem solving and design for the human engineer. Engineering speed and quality should improve as more engineering test and validation activities become automated. The idea of “cognitive assistants” that broadly support the engineer will evolve but they must evolve in a way that supports the problem solving and associated learning processes associated with engineering [14, 23].

4. CONCLUSIONS

This chapter presented four new types of systems that collectively are driving significant change in the SE community. These systems are highly adaptive, are highly connected including CPS and other systems that are the traditional core of SE, are governed in a highly distributed and flexible manner, and increasingly learn new behaviors and adapt on their own.

Seven trends that will significantly change the SE discipline are discussed, primarily from research and road mapping activities over the past 5 years in the SERC. These are digitalization, agile processes, the evolving digital SE ecosystem, modularity and open business models, digital system models, ontologies and semantic technology, and, of course, AI and ML. Many of these trends are occurring independently of each other, and it is the role of SE to be the integrator of these as well as the systems we impact. Some of these will be further elaborated in the following chapters.

There are numerous other trends that have not been included in this chapter but may also need to be incorporated into the evolution of SE practice. These have been chosen because, in our experience, are having the strongest impact in SE transformation at the time of writing this chapter.

REFERENCES

1. Acquisition Innovation Research Center (AIRC) (2023). Agile Development of Hardware-Reliant Systems. Retrieved September 1, 2023 from <https://acqirc.org/events/agile-development-of-hardware-reliant-systems-workshop/>.
2. Bhatia H (2018). "Connected car opportunity propels multi-billion-dollar turf war." Economic Times. Retrieved September 1, 2023 from <https://telecom.economictimes.indiatimes.com/tele-talk/connected-car-opportunity-propels-multi-billion-dollar-turf-war/2971>.
3. Boardman J and Sauser B (2006). "System of systems - The meaning of 'of,'" In 2006 IEEE/SMC International Conference on System of SE. IEEE.
4. Brose C (2020). The Kill Chain: Defending America in the Future of High-Tech Warfare. Hachette Books.
5. Brunton S et al. (2020). Data-Driven Aerospace Engineering: Reframing the Industry with Machine Learning. Retrieved September 1, 2023 from https://www.researchgate.net/publication/343877323_Data-Driven_Aerospace_Engineering_Reframing_the_Industry_with_Machine_Learning.
6. Delligati L (2013). SysML Distilled: A Brief Guide to the Systems Modeling Language. Addison-Wesley.
7. Dahmann J, Rebovich G, Lane J, Lowry R, and Baldwin K (2011), "An implementers' view of SE for systems of systems," 2011 IEEE International Systems Conference, Montreal, pp. 212-217.
8. Geels F (2002). "Technological transitions as evolutionary reconfiguration processes: A multi-level perspective and a case-study," Research Policy, December 2002, 31 (8-9), 1257-1274.
9. Gorman S & Eiras A (2023). "SpaceX rocket explosion illustrates Elon Musk's 'successful failure' formula". Reuters. Retrieved September 1, 2023 from <https://www.reuters.com/lifestyle/science/spacex-rocket-explosion-illustrates-elon-musks-successful-failure-formula-2023-04-20/>.
10. Griffor E (ed) (2016). Handbook of System Safety and Security: Cyber Risk and Risk Management, Cyber Security, Adversary Modeling, Threat Analysis, Business of Safety, Functional Safety, Software Systems, and Cyber Physical Systems. Syngress.
11. Griffor, E, Greer, C, Wollman, D and Burns, M (2017), Framework for Cyber-Physical Systems: Volume 1, Overview, Special Publication (NIST SP), National Institute of Standards and Technology, Gaithersburg, MD, [online], <https://doi.org/10.6028/NIST.SP.1500-201>. Retrieved September 1, 2023.
12. INCOSE (2023). SE Vision 2035: Engineering Solutions for a Better World. International Council on SE. Retrieved September 1, 2023 from <https://www.incose.org/about-systems-engineering/se-vision-2035>.
13. Maier M (1996). "Architecting Principles for Systems of Systems," Proc. of the Sixth Annual International Symposium, International Council on SE, Boston, MA, pp. 567- 574.

14. McDermott, T, Blackburn, M, & Beling, P (2021). Artificial Intelligence and Future of SE. In: Lawless, W.F., Mittu, R., Sofge, D.A., Shortell, T., McDermott, T.A. (eds) SE and Artificial Intelligence. Springer, Cham. https://doi.org/10.1007/978-3-030-77283-3_3.
15. McDermott, T, DeLaurentis, D, Beling, P, Blackburn, M & Bone, M (2020), AI4SE and SE4AI: A Research Roadmap. INSIGHT, 23: 8-14. <https://doi.org/10.1002/inst.12278>.
16. McDermott T, Alexander K, & Wallace R (2023). The Supra-System Model. INSIGHT 26 (2), 15-21.
17. McDermott and Horowitz 2017 is "McDermott T, Horowitz B (2017) Human Capital Development – Resilient Cyber Physical Systems. Systems Engineering Research Center (SERC) Technical Report SERC-2017-TR-075, September 29, 2017.
18. National Academies of Sciences, Engineering, and Medicine (2016). A 21st Century Cyber-Physical Systems Education. Washington, DC: The National Academies Press. doi:10.17226/23686.
19. National Science Foundation (NSF) (2016). Cyber Physical Systems (CPS). Retrieved September 1, 2023 from <https://new.nsf.gov/funding/opportunities/cyber-physical-systems-cps>.
20. Office of the Deputy Assistant Secretary of Defense for SE (2018). Department of Defense Digital Engineering Strategy, Washington DC.
21. Rotmans J, Kemp R, & van Asselt M (1999). "More evolution than revolution: transition management in public policy," Foresight 3 (1).
22. SAE (2023). OnQue™ Digital Standards System. Retrieved September 1, 2023 from <https://www.sae.org/onque-digital-standards>.
23. Salado A and Selva D (2021). "Asistentes Cognitivos en Ingeniería," UE Essentials, Universidad Europea de Madrid, Madrid, Spain.
24. Systems Engineering Research Center (SERC) (2017). Human Capital Development – Resilient Cyber Physical Systems. Technical Report SERC-2017-TR-075. Stevens Institute of technology, Hoboken NJ.
25. SERC (2023). Information Models and Ontologies to Enable Digital Engineering. Retrieved September 1, 2023 from <https://sercuarc.org/event/information-models-and-ontologies-to-enable-digital-engineering-research-workshop/>.
26. Shadab N, Kulkarni A, Salado A (2021). "Challenges to the Verification and Validation of AI-enabled Systems: A Systems-Theoretic Perspective," in W.F. Lawless, R. Mitty, D. Sofge, T. Shortell, T. McDermott (Eds.), Systems Engineering and Artificial Intelligence, (pp. 363-378), Cham: Springer, 2021.
27. von Bertalanffy L. (1969). General Systems Theory. New York: George Braziller.
28. Wikipedia contributors. (2021, September 29). Adaptability. In Wikipedia, The Free Encyclopedia. Retrieved September 1, 2023, from <https://en.wikipedia.org/w/index.php?title=Adaptability&oldid=1047218141>.
29. Zhu H and Arnold E, "System Adaptability." in SEBoK Editorial Board. 2023. The Guide to the SE Body of Knowledge (SEBoK), v. 2.8, R.J. Cloutier (Editor in Chief). Hoboken, NJ: The Trustees of the Stevens Institute of Technology. Accessed September 1, 2023. www.sebokwiki.org. BKCASE is managed and maintained by the Stevens Institute of Technology SE Research Center, the International Council on SE, and the Institute of Electrical and Electronics Engineers Systems Council.

BIOGRAPHIES

THOMAS ALLEN MCDERMOTT

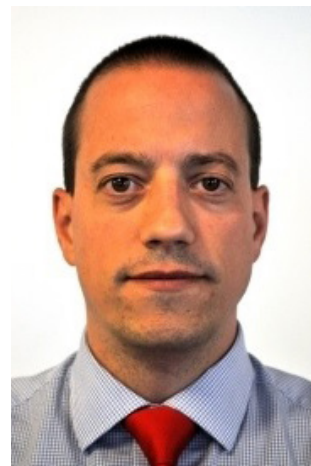
Tom McDermott is the Chief Technology Officer of the Systems Engineering Research Center (SERC) and a faculty member in the School of Systems and Enterprises at Stevens Institute of Technology in Hoboken, NJ. With the SERC he develops new research strategies and is leading research on digital transformation, education, security, and artificial intelligence applications. He previously held roles as Faculty and Director of Research at Georgia Tech Research Institute and Director and Integrated Product Team Manager at Lockheed Martin.



Mr. McDermott teaches system architecture, systems and critical thinking, and engineering leadership. He provides executive level consulting as a systems engineering and organizational strategy expert. He is a fellow of the International Council on Systems Engineering (INCOSE) and recently completed 3 years as INCOSE Director of Strategic Integration.

VÍCTOR RAMOS DEL POZO

Víctor Ramos del Pozo is a systems engineer at Isdefe, currently delivering technical assistance for the Spanish National Programme Office for the Next Generation Weapon System (NGWS) within a Future Combat Air System (FCAS), especially in the fields of the Combat Cloud, Collaborative Sensors and Simulation. He previously delivered technical assistance for the Spanish National Armaments Directorate in the fields of defence industrial base analysis, defence acquisition planning and major defence acquisition programmes management.



Before joining Isdefe, he worked as a systems engineer at EADS-CASA (current Airbus Defence and Space) for the Multi-Role Tanker Transport and Future Strategic Tanker Aircrafts programmes, and at Indra for the Identification Friend or Foe Department.

He is a member of the International Council on Systems Engineering (INCOSE) and delivers internal training at Isdefe in the fields of systems engineering and programme management.



"We have developments that follow all of the rules, but fail."

J. Frosch

Evolution of systems engineering development and execution models

Dr. Ronald Giachetti, *Naval Postgraduate School (regiache@nps.edu)*
Juan Carlos Lario Monje, *Isdefe (jclario@isdefe.es)*

Abstract

This chapter presents the need to evolve and adapt systems engineering development models to the current context of engineering projects. The chapter separates the discussion between traditional, dominantly plan-driven approaches to systems development such as the Waterfall and Vee models, and discussion of agile development approaches that emphasize responsiveness and are characterized as highly iterative and incremental. Most projects could benefit from both, and we discuss hybrid approaches and how to tailor the development models to the context of particular projects. The chapter discusses other emergent methods including the merging of development with operations through DevOps.

Keywords

Systems engineering, development models, agile, DevOps, hybrid usage.



1. INTRODUCTION

A system development model defines an approach to develop a system to satisfy some needs expressed by one or more stakeholders. The development of large scale and complex systems often entails multiple technologies and a large number of diverse individuals bringing different knowledge and skills to the effort. This chapter first describes the purpose and intent of system development models. The models are organized into two categories: plan-driven models and agile models. Plan-driven models assume we sufficiently understand the needs and solution such that we can plan the development project in advance and then develop the system according to the plan. Agile models take the position that we do not fully understand the needs or solution and therefore the project execution must be adaptive or flexible, allowing for changes in the needs and/or the solution. After introducing multiple representative development models, the chapter then discusses how to tailor a development model to a particular project considering all the contextual factors contributing to a successful outcome.

1.1. System development context

The development of large-scale, complex systems places multiple demands of organizational nature (i.e., over the organization that develops the system, also called the realization system), which a system development model is intended to address. In this sense, the basic demands a team developing a new system will have to face are understanding the stakeholder needs, creating a concept addressing those needs, designing and integrating the concept with hardware, software, processes, and people, and then implementing the designed system so that it can be deployed.

System development is almost always constrained by schedule and budget, and system development models must be realizable in these conditions. An organization engages the work of a large number of people with different knowledge and skills during the system development, all of whom must be organized in an effective way to design and build the system. A development model provides a means to understand how to organize the work of all these individuals. Many systems include new or emerging technologies, which introduce risk into the development. Under these conditions, a system development model must provide for means to mature such technologies and to assess the associated risks to the development organization as well as to the system itself (e.g., risks that may materialize during system operation).

Many systems, such as those incorporating greater autonomy or artificial intelligence, face the challenge of maturing a technology in parallel to system development. At the same time, safety-critical systems, such as aircraft or medical devices, must be developed and will operate in highly regulated environments. These aspects generally impose severe constraints on the generation of information and data during system development, which is enabled and/or facilitated by adopting a suitable development model.

Since software has become an essential component of most systems, system development must now contend with the generally high complexity introduced by software, potential security vulnerabilities previously unaccounted for, and rapid deployment at *almost the click of a button*. In the past, when systems consisted mostly of hardware components, the system capabilities generally remained unchanged once the system was deployed. Because software is not physical (it is a set of instructions that must be installed on hardware), it enables system owners to continuously update system capabilities during utilization. Furthermore, software components have also enabled the implementation of artificial intelligence, which can create scenarios in which the algorithms and/or decisions made by the system can change over time [26]. This creates challenges for system verification as well as for human trust in the system concerning safety as well as other issues. Collectively, these characteristics affect the development process of software-intensive systems and any software components found in larger systems.

This section has described the context surrounding modern large-scale and/or complex system development projects. System development models have evolved to contend with many of the issues presented above.

1.2. Definition of a system development model

A system development model provides a **structured framework** and set of guidelines for organizing the development activities and coordinating the technical development of a system. Development models are often governed by a set of principles and/or best practices. The use of a development model helps in organizing and managing the associated efforts, ensuring a certain quality level, reducing risks, and improving collaboration and communication among team members and other stakeholders.

Following a defined process is correlated with a higher likelihood of delivering successful system and capabilities within the constraints imposed on time, cost, and quality [7].

System development models generally divide the process into phases to describe the progress or evolution of the system through its life cycle. Milestones are *decision gates* marking the completion of a phase, at which time the development organization decides whether to continue the system development to the next phase, to wait for a complete accomplishment of certain identified pending actions, or, as a worst case, terminate the project. Entry and exit criteria for each milestone are generally well defined in advance in order to minimize any undesirable uncertainties. In this sense, the risk of going ahead is often estimated and weighed before making the decision.

1.3. System life cycle model

The development of a system must be understood in the context of the system life cycle. A system life cycle describes the phases a system goes through from its initial conceptualization to its retirement or disposal, of which development is just one portion. Figure 1 shows an example of how ISO/IEC 15288 defines them. The arrows indicate the existence of multiple potential paths through the different phases, which may involve iteration and repetition. A new, unprecedented system might go through the phases sequentially from concept to development, to production, and to utilization and support, before being retired. Instead,

an existing system might be upgraded and cycled back from utilization to concept again. Some systems may be designed and deployed incrementally and would transit through multiple iterations of the phases for different aspects of the overall system.

It should be noted that systems engineering activities such as the elicitation of stakeholder needs or system integration are not *exclusively* associated with any of the individual phases of the system lifecycle. Furthermore, some system engineering activities may be executed iteratively and recursively at multiple levels of the system hierarchy. Consequently, systems engineering spans all the phases of a system's life cycle.

1.4. System development models

As the body of knowledge surrounding process models evolves, the International Council on Systems Engineering (INCOSE) has revised their classification of system development models several times [20]. We describe two categories as:

- 1) Pre-specified or plan-driven models.
- 2) Evolutionary or agile models.

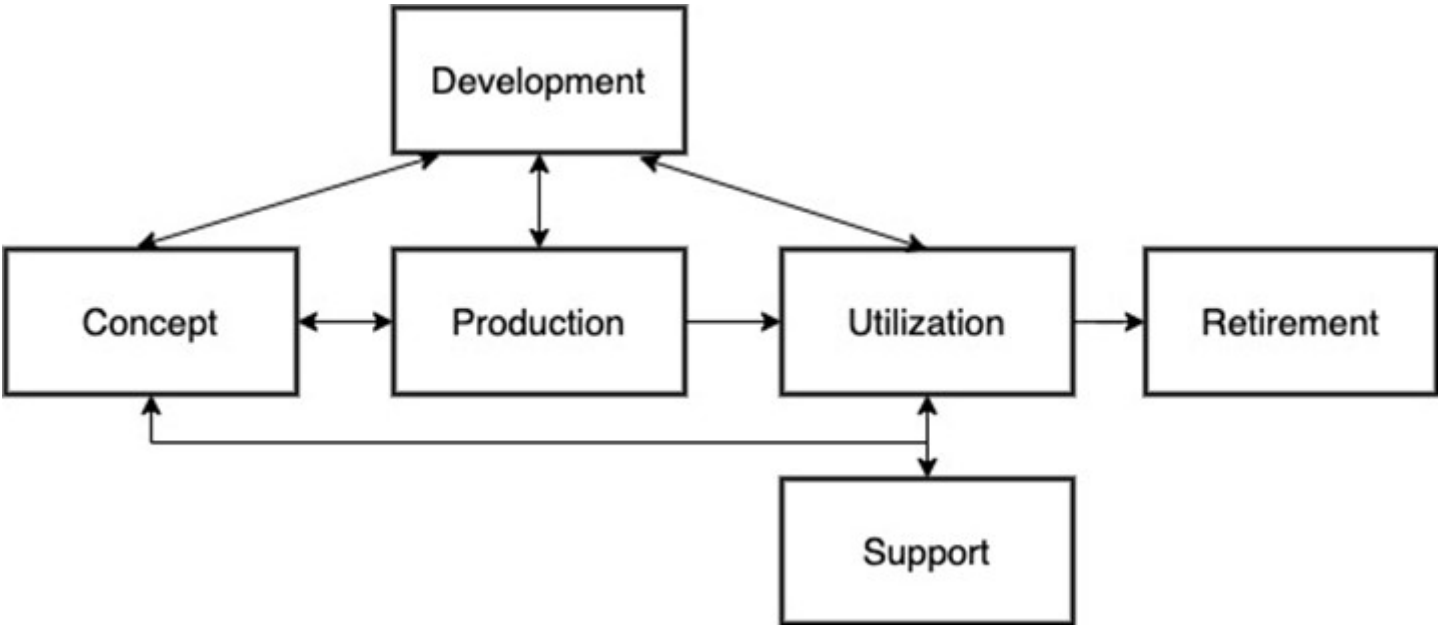


Figure 1. System life cycle (adopted figure from INCOSE Handbook, 2023)

The first category includes traditional models such as the *Waterfall model* and the *Vee model*. The second category describes an evolution away from strictly plan-driven models and includes the *Spiral model* based on iterative development cycles. The second category has come to be called agile methods and borrows heavily from the software engineering community. The categorization is according to the *dominant* characteristics of the corresponding models. Nevertheless, it should be noted that certain overlap between the categories exist because most models share characteristics. For instance, many authors present the Waterfall model as the archetypical plan-driven model with strictly sequential execution of activities. Yet, the original article by Royce acknowledged iteration among the activities. In fact, iteration and recursion are always inherent to any development process, within and between its different phases.

2. PLAN-DRIVEN DEVELOPMENT MODELS

Plan-driven models describe a class of system development models that prioritize the predictability available through adhering to a plan laying out a structured development approach. This section reviews the archetypical plan-driven development models of the Waterfall and the Vee models.

2.1. Waterfall model

The Waterfall model is a plan-driven model in which all engineering activities are defined and planned at the start of the project. Each activity in the Waterfall model is supposed to be executed completely before the next activity begins. Moreover, the output of the preceding activity becomes input to the next activity. Iteration of activities is not planned in the Waterfall model but is expected to happen only when problems are not fixable at one stage and, in such a case, the project *moves* just to the previous stage. Figure 2 shows the obvious *sequential* structure of the activities in the Waterfall model.

Although the Waterfall model is often criticized and almost considered irrelevant in systems engineering circles these days, the model is still well suited and useful for projects in which:

- it is possible to confidently know the requirements at the beginning,
- those requirements will be stable and unchanging,
- there is little technical risk, and/or
- there are hazardous risks that require careful progression of activities.

Under these conditions, the Waterfall model's sequential and structured approach for system development provides the benefits of predictability and easier coordination of the work activities.

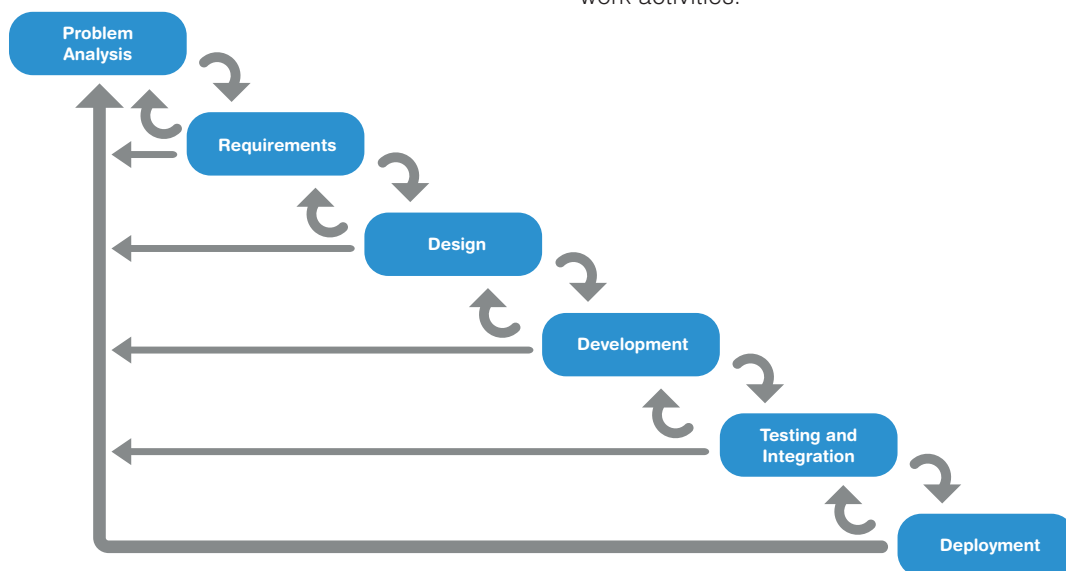


Figure 2. Waterfall model